

Backend URI and Service Flow

Project: **American Latin Class / ALCSys**tem

Date: **June 10, 2026**

Purpose

This document explains how the backend URIs are connected to controllers, internal services, models, and database tables. It is intended for the backend URI presentation. It also clarifies an important architecture point:

- The project is **not a microservices system**.
- The project is a **modular REST API backend**.
- The backend uses **internal services** to keep business logic separated from controllers.

URI, Controller, and Service Relationship

A URI is the external address used by the frontend or an API client.

Example:

POST /api/auth/login

That URI is mapped in:

06Code/backend/routes/api.php

The route calls a controller method. The controller may call one or more internal services. The services may use models, and the models communicate with Supabase PostgreSQL through Eloquent ORM.

General flow:

Frontend or API client

- > Backend URI
- > routes/api.php
- > Controller method
- > Internal service or services
- > Eloquent model or models
- > Supabase PostgreSQL table or tables
- > JSON response

Is This Microservices?

No. A microservices system normally has separate deployable services, for example:

auth-service
attendance-service
finance-service
student-service

Each service may have its own deployment, API, database, and network communication with other services.

This project has one backend application deployed as one API service on Render. Inside that application, the code is divided into smaller internal services such as **AuthService**, **BranchAccessService**, **TeacherPayrollService**, and **ValidationService**.

So the correct description is:

Single REST API backend with modular internal services.

Main Backend Components

Component	Location	Responsibility
Route table	06Code/backend/routes/api.php	Connects each URI to a controller method and applies middleware.
Controllers	06Code/backend/src/Controllers	Receive requests, coordinate services/models, and return JSON.
Services	06Code/backend/src/Services	Hold reusable business rules such as auth, validation, payroll, dates, branch access, and audit.
Middleware	06Code/backend/src/Middleware	Protects routes using Bearer token and role checks.
Models	06Code/backend/src/Models	Map database tables using Eloquent ORM.
Support	06Code/backend/src/Support	Handles JSON responses, CORS, and database bootstrapping.
Database	Supabase PostgreSQL	Stores branches, students, users, attendance, finance, events, and audit data.

Internal Services

Service	Main Responsibility	Used By
AuthService	Validates credentials, creates public user payloads, reads users from Bearer token.	AuthController, RoleMiddleware
JwtTokenService	Issues and verifies signed tokens.	AuthService
BranchAccessService	Applies branch permissions and write scopes.	Student, teacher, attendance, finance, event controllers
DateRangeService	Converts a month into a start/end date range.	Auth, student, attendance controllers
AttendanceSummaryService	Counts attendance totals, present, late, absent, and excused records.	Auth and student controllers
TeacherPayrollService	Calculates teacher late/present status and payroll summaries.	Teacher attendance and attendance controllers
EvidenceCodeGenerator	Generates attendance evidence codes.	Attendance, kiosk, teacher attendance controllers
AuditLogger	Records protected write actions when <code>audit_logs</code> is available.	Student, teacher, class plan, attendance, finance, event controllers
ValidationService	Facade used by controllers; delegates request checks to focused validators under <code>Services/Validation</code> .	Most write controllers
Services/Validation/*	Domain validators for students, teachers, attendance, planning, finance, events, profile photos, common fields, and Ecuadorian IDs.	ValidationService

Public and Diagnostic URIs

URI	Controller method	Services used	Models / tables	Response purpose
GET /	HomeController::index	None	None	Backend metadata and endpoint summary.
GET /api/health	HomeController::health	None	Database connection only	API and database health status.
GET /api/debug	HomeController::debug	None	Database connection only	Development diagnostic environment/database status. Registered only when <code>APP_DEBUG=true</code> .
GET /api/branches	BranchController::index	None	Branch / branches	Public branch list with fallback data.
GET /api/styles	ReferenceDataController::index	None	DanceStyle / dance_styles	Public dance style list with fallback data.
GET /api/levels	ReferenceDataController::index	None	Level / levels	Public student level list with fallback data.

Enrollment and Authentication URIs

URI	Controller method	Services used	Models / tables	Response purpose
POST /api/enrollments	EnrollmentController::validate	ValidationService	Branch, Student / branches, students	Creates a pending enrollment request.
POST /api/auth/login	AuthController::login	AuthService, JwtTokenService through AuthService	User / users	Returns signed token and user data.
POST /api/auth/google	AuthController::googleLogin	AuthService, JwtTokenService through AuthService	User, Student / users, students	Logs in with Google or tells frontend that enrollment is needed.
POST /api/auth/google/register	AuthController::googleRegister	AuthService, JwtTokenService through AuthService	User / users	Creates a student user from Google token only.
POST /api/auth/google/enroll	AuthController::googleEnroll	AuthService, JwtTokenService through AuthService	Branch, Student, User / branches, students, users	Creates active student and linked user after Google verification.

Attendance Station URIs

URI	Controller method	Services used	Models / tables	Response purpose
POST /api/kiosk/attendance	KioskController::store	ValidationService, EvidenceCodeGenerator	Student, AttendanceRecord / students, attendance_records	Registers legacy student kiosk attendance by national ID.
POST /api/teacher-attendance/check-in	TeacherAttendanceController::checkIn	ValidationService, EvidenceCodeGenerator, TeacherPayrollService	User, Branch, AttendanceRecord / users, branches, attendance_records	Registers teacher check-in, status, pay rate, and evidence code.

Authenticated User URIs

URI	Controller method	Middleware	Services used	Models / tables	Response purpose
GET /api/me	AuthController::me	RoleMiddleware: teacher, student, director	AuthService, DateRangeService, AttendanceSummaryService	User, Student, AttendanceRecord, students, attendance_record	Returns current user profile; students also receive attendance summary.
GET /api/me/attendance	StudentController::me	RoleMiddleware: student	DateRangeService, AttendanceSummaryService	AttendanceRecord, attendance_record	Returns current student monthly attendance.
PATCH /api/me/photo	ProfilePhotoController::me	RoleMiddleware: student	ValidationService	User, Student / users, students	Updates current student profile photo.

Director Student and Teacher URIs

URI	Controller method	Middleware	Services used	Models / tables	Response purpose
GET /api/students	StudentController::index	RoleMiddleware: director	BranchAccessService	Student, Branch / students, branches	Lists students visible to the director.
POST /api/students	StudentController::create	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	Branch, Student, AuditLog / branches, students, audit_logs	Creates a student and records audit metadata.
PATCH /api/students/{studentId}	StudentController::update	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	Student, AuditLog / students, audit_logs	Updates student data if branch access allows it.
DELETE /api/students/{studentId}	StudentController::destroy	RoleMiddleware: director	BranchAccessService, AuditLogger	Student, AuditLog / students, audit_logs	Deactivates a student.
GET /api/teachers	TeacherController::index	RoleMiddleware: director	BranchAccessService	User / users	Lists teacher accounts visible to the director.
POST /api/teachers	TeacherController::create	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	Branch, User, AuditLog / branches, users, audit_logs	Creates a teacher account.
PATCH /api/teachers/{teacherId}	TeacherController::update	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	User, AuditLog / users, audit_logs	Updates teacher account data.
DELETE /api/teachers/{teacherId}	TeacherController::destroy	RoleMiddleware: director	BranchAccessService, AuditLogger	User, AuditLog / users, audit_logs	Deactivates a teacher account.

Planning and Attendance Record URIs

URI	Controller method	Middleware	Services used	Models / tables	Response purpose
GET /api/class-plans	ClassPlanController	RoleMiddleware: teacher, director	BranchAccessService	ClassPlan / class_plans	Lists class plans by teacher or director branch scope.
POST /api/class-plans	ClassPlanController	RoleMiddleware: teacher, director	BranchAccessService, ValidationService, AuditLogger	Branch, ClassPlan, AuditLog / branches, class_plans, audit_logs	Submits a class plan.
GET /api/attendance-records	AttendanceRecordController	RoleMiddleware: teacher, director	DateRangeService, BranchAccessService, TeacherPayrollService	AttendanceRecord	Lists attendance records and teacher payroll summary.
POST /api/attendance-records	AttendanceRecordController	RoleMiddleware: teacher, director	BranchAccessService, ValidationService, EvidenceCodeGenerator, AuditLogger	Student, Branch, AttendanceRecord, AuditLog / students, branches, attendance_records, audit_logs	Creates manual student or teacher attendance record.

Finance and Professional Event URIs

URI	Controller method	Middleware	Services used	Models / tables	Response purpose
GET /api/branch-finance-reports	FinanceController	RoleMiddleware: director	BranchAccessService	BranchFinanceReport / branch_finance_reports	Lists branch finance reports.
POST /api/branch-finance-reports	FinanceController	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	Branch, BranchFinanceReport, AuditLog / branches, branch_finance_reports, audit_logs	Creates finance report and calculates matrix share/net result.
GET /api/professional-events	ProfessionalEventController	RoleMiddleware: director	BranchAccessService	ProfessionalEvent, DancerEventAssignment / professional_events, dancer_event_assignments	Lists B2 professional events and assignments.
POST /api/professional-events	ProfessionalEventController	RoleMiddleware: director	BranchAccessService, ValidationService, AuditLogger	Branch, ProfessionalEvent, AuditLog / branches, professional_events, audit_logs	Creates B2 professional event.
POST /api/professional-events/{eventId}/assignments	ProfessionalEventController	RoleMiddleware: assignDancer	BranchAccessService, ValidationService, AuditLogger	ProfessionalEvent, Student, DancerEventAssignment, AuditLog / professional_events, students, dancer_event_assignments, audit_logs	Assigns a B2 dancer to a professional event.

URI	Controller method	Middleware	Services used	Models / tables	Response purpose
GET /api/dancer-settlements/{studentId}	ProfessionalEventController::settlement()	RoleMiddleware, BranchAccessService	Student, DancerEventAssignment, ProfessionalEvent	Students, dancer_event_assignment, professional_event	Calculates gross, deductions, net amount, and assignment history for one B2 dancer.

Example Flow: Login

```
POST /api/auth/login
-> routes/api.php
-> AuthController::login()
-> AuthService::attempt()
-> User model
-> AuthService::issueToken()
-> JwtTokenService::issue()
-> JSON response with token and user
```

Example Flow: Student Monthly Attendance

```
GET /api/me/attendance?month=2026-06
-> RoleMiddleware validates Bearer token and student role
-> StudentController::attendance()
-> DateRangeService::month()
-> AttendanceRecord model
-> AttendanceSummaryService::fromRecords()
-> JSON response with month, summary, and records
```

Example Flow: Teacher Check-In

```
POST /api/teacher-attendance/check-in
-> TeacherAttendanceController::store()
-> ValidationService::validateAttendanceTeacherKiosk()
-> User model checks active teacher
-> Branch model checks branch
-> TeacherPayrollService::attendanceStatus()
-> EvidenceCodeGenerator::makeAttendanceCode()
-> AttendanceRecord model creates teacher_kiosk record
-> JSON response with evidence code and status
```

Example Flow: B2 Event Settlement

```
GET /api/dancer-settlements/{studentId}
-> RoleMiddleware validates director role
-> ProfessionalEventController::settlement()
-> BranchAccessService::applyScope()
-> Student model checks B2 dancer
-> DancerEventAssignment model loads event assignments
-> Controller calculates gross, deductions, and net amount
-> JSON response with settlement summary
```

Summary for Presentation

The backend URIs are the public contract of the API. Each URI is mapped to a controller. Controllers coordinate internal services and models. Services keep reusable business logic separate from HTTP code. Models connect the application to Supabase PostgreSQL.

This proves the project is separated by responsibility, even though it is not a microservices architecture.